

Feature: CRM spine

From build-plan: feature 4

Branch: feature/04-crm-spine

Status: verified. Whole feature seen and agreed by Frank 2026-09-30; steps 4.1 to 4.3 built, tested and reviewed step by step; every review finding fixed, F-56 last on 2026-10-01. The checkpoint for the final review.

Branch named in the workspace's `feature/NN-name` form, so one number finds the branch, the archive (`04-crm-spine.md`) and the tag (`item-04-done`).

Goal

The three tables every later module writes to, built before anything reads them, so the CRM fits later without a rewrite (project plan, decision 5):

- `pipeline_stage` : a business's own ordered stages. Every business has the four defaults, New, Contacted, Booked and Done: existing businesses get them by migration, new ones when they are set up.
- `contact` : the people a business deals with, one timeline each.
- `activity` : one table for the timeline (what happened) **and** the next-step queue (what is still owed), the Pipedrive idea.

With them, the small backend functions the booking loop (features 5 to 8) calls to use them: the business's first stage, find or create a contact, and record a timeline entry. Nothing is visible yet; the screens are feature 11.

In scope

- The three tables, each with its migration, organization-scoped like every app table, and the database rules that keep one business's rows away from another's.
- The four default stages: a migration backfill for every existing business, the `afterCreateOrganization` hook for every new one (so `POST /admin/clients` gives a client their stages), and the dev seed.
- Backend functions in `backend/lib/crm/` : `seedPipelineStages` , `findFirstPipelineStage` , `findOrCreateContact` , `recordActivity` .
- The shared lists both sides will read: the default stage names and the activity types.
- Tests for every function and every database rule, against the local database.

Out of scope

- **HTTP routes.** See Notes for the AI and Decisions: the build plan's "API routes the loop writes to" is read as the server-side functions the loop calls. Routes arrive with the screens that need them (feature 11 for leads, contacts and next steps, feature 12 for stages).
- `lead` , `booking` , `commitment` : feature 5. A stage is pointed at by a lead, which does not exist yet.
- Renaming, reordering, adding or deleting stages: features 12 and 14.
- Completing a next step, listing a timeline, adding a contact by hand: the screens of feature 11. The table already holds next steps; nothing writes one yet.
- Merging duplicate contacts, importing contacts, contact search.
- Any screen.

Build loop

`workflow.stepReview` is `every` and `workflow.checkpointCommits` is `enabled` .

Each step's plan (Part 1, what it builds; Part 2, Done when) is gone through with Frank and gets its own yes just before it is built. After that yes the step runs straight through without asking: build, tests and checks, tick the box here, write the build log entry and push buildlogs, commit (`feat: 4.N <what>`) and push to the feature branch, `/audit` scoped to the step, then the independent review. The one planned stop is after the review, where its findings are talked through. P0/P1 are fixed before the next step, or Frank accepts them with a reason; P2/P3 are recorded and carried.

Only three things stop a step earlier: the agreed plan turns out wrong while building, a line only Frank crosses (a package, Railway or real data, `main` , a merge, a force push, deleting anything), or blocking review findings.

`/complete` runs its own final review over the already-reviewed steps, then merges with a merge commit on Frank's yes.

Build steps

- ☒ **4.1 Pipeline stages.** Every business has its four stages.
 - **Plan approved by Frank 2026-09-30, with one tightening:** stage names are unique in a business ignoring capitals ("New" and "new" are the same name), a unique index on `(organizationId, lower(name))` .
 - `packages/shared/crm/default-pipeline-stages.ts` : the four, in order: New, Contacted, Booked, Done. A new code folder: added to both `tsconfig` include lists and given its own subpath export (`@scheduleads-app/shared/crm`), as the standards require.
 - `packages/shared/db/crm-tables/pipeline-stage-table.ts` and migration `0006_pipeline_stage` (generated, then a hand-written backfill at its end that gives every existing

business the four, as 0004's backfill did).

- `backend/lib/crm/seed-pipeline-stages.ts` : gives a business the four if it has none; running it twice changes nothing.
- `backend/lib/crm/find-first-pipeline-stage.ts` : the business's first stage, where feature 5 puts a new lead.
- The `afterCreateOrganization` hook calls `seedPipelineStages` after the first person. If it fails, `provision-client.ts` removes the half-made business as it already does.
- `packages/shared/scripts/seed-dev.ts` gives both dev businesses the four.
- **Done when** `npm run test --workspace=backend` and the shared tests pass, with tests proving: a business set up through `POST /admin/clients` has exactly New, Contacted, Booked, Done in that order; running the seed function again adds nothing; `findFirstPipelineStage` gives New, and nothing for a business with no stages; two businesses can both have a stage named New, one business cannot have two, nor "New" and "new"; and after `db:migrate` and `db:seed`, Summit Painting (dev) and Riverbend Clinic (dev) each have the four. The backend builds.
- ☒ **4.2 Contacts.** The people a business deals with.
 - **Blocker, answered by Frank 2026-10-01: A.** The same email in the same business is the same contact (a repeat customer's bookings land on one timeline); no email is always a new contact; the first name given is kept. Rejected, B: every booking a new contact (a split history, and merging by hand is a screen no feature plans).
 - `packages/shared/db/crm-tables/contact-table.ts` and migration `0007_contact`.
 - `packages/shared/zod-validation/crm-validation-schemas/contact-validation-schema.ts` : name (trimmed, 1 to 120), email (the shared email rule, optional), phone (trimmed, up to 40, optional).
 - `backend/lib/crm/find-or-create-contact.ts`, the function feature 5 calls for every booking.
 - **Plan approved by Frank 2026-10-01, with one tightening (3b's F-41 lesson):** no customer email in an error. A database error's message carries the whole query, the email in it, and Hono logs what reaches it; `findOrCreateContact` lets only a safe reason out (`safeErrorReason`). Tested by forcing a real database error and checking the thrown message.
 - **Carried from step 4.1's review (Frank, 2026-10-01), F-53:** the capitals test in `seed-pipeline-stages.test.ts` seeds its own business and expects the refusal to come from `pipeline_stage_organization_name_unique` (Postgres code 23505), not any failed insert.
 - **Done when** the backend and shared tests pass, with tests proving the matching rule Frank picks; that a contact is only ever found inside its own business (the same email in two businesses is two contacts); that two calls at the same moment with the same email make one contact; that an email is stored lowercased; and that a contact with no email is always a new one. The backend builds.
- ☒ **4.3 The activity table: timeline and next steps.**
 - `packages/shared/crm/activity-types.ts` : the eight types, `booking_created`, `stage_changed`, `email_sent`, `email_received`, `note`, `sms_sent`, `call`, `task`, one list both sides import.

- `packages/shared/db/crm-tables/activity-table.ts` and migration `0008_activity`.
- `backend/lib/crm/record-activity.ts` : records something that happened on a contact's timeline.
- **Plan approved by Frank 2026-10-01, with two tightenings:** (1) the same no-personal-data-in-an-error rule as contacts, because a payload can hold a customer's words: `recordActivity` lets only a safe reason out; (2) a test saves one row of every shared type and expects all eight accepted, so the shared list and the database's check cannot drift apart.
- **Done when** the backend tests pass, with tests proving: an entry is recorded with its type, payload, actor and time; an entry cannot name another business's contact (refused by the database, not only the code); a row is either something that happened (`occurredAt`) or something still to do (`dueAt` , later `doneAt`), never both and never neither; a type outside the list is refused; and deleting a contact takes its timeline with it. The backend builds.

Files / areas

- `packages/shared/crm/` (new): `default-pipeline-stages.ts` , `activity-types.ts` , tests beside them; `packages/shared/package.json` (the `./crm` export), `tsconfig.json` and `tsconfig.build.json` (the folder)
- `packages/shared/db/crm-tables/` (new): one table per file; `packages/shared/db/index.ts`
- `packages/shared/migrations/0006` to `0008` , generated from `packages/shared` only
- `packages/shared/zod-validation/crm-validation-schemas/` (new) and `zod-validation/index.ts`
- `packages/shared/scripts/seed-dev.ts`
- `backend/lib/crm/` (new): one function per file, tests beside them
- `backend/lib/auth/auth-server.ts` (the hook)

Data / contracts

Every table carries `organizationId` (FK to `organization` , cascade), text ids (`randomUUID()`), `createdAt` and `updatedAt` as `timestamptz` . The business always comes from the caller on the server (a session, or the slug lookup of a public route), never from anything a client sends; every function takes `organizationId` as its first argument and every query filters on it.

pipeline_stage

- `id` , `organizationId` , `name` (text, not null), `position` (integer, not null).
- Unique (`organizationId` , `lower(name)`) : one business cannot have two stages with the same name, capitals ignored (Frank, 2026-09-30). Unique (`organizationId` , `id`) , so feature 5's `lead` can point at a stage of its own business only, as `availability_rule` does with `resource` .

- Order is `position` ascending, ties broken by `createdAt` then `id`. `position` is not unique, so reordering later (features 12, 14) needs no juggling.
- Defaults: New 1, Contacted 2, Booked 3, Done 4. "First stage" means the lowest position.

contact

- `id`, `organizationId`, `name` (text, not null), `email` (text, nullable, always stored lowercased: a check refuses anything else), `phone` (text, nullable, stored as typed, trimmed).
- Unique `(organizationId, id)`, so `activity` (and feature 5's `lead`) can point at a contact of its own business only.
- Matching (Frank, 2026-10-01): unique `(organizationId, email)` where `email` is not null. `findOrCreateContact(organizationId, { name, email?, phone? })` returns `{ contact, created }`: with an email it inserts with `on conflict do nothing` and then reads the row, so two bookings at the same moment cannot make two contacts; the existing contact's name and phone are kept, not overwritten. Without an email it always creates.
- No rule that a contact must have an email or a phone: the forms that make contacts (features 5 and 11) decide what they require.

activity

- `id`, `organizationId`, `contactId` (not null; FK `(organizationId, contactId)` to `contact(organizationId, id)`, cascade), `type` (text, not null, checked against the eight types), `payload` (jsonb, not null, default `{}`), `actorUserId` (nullable, FK `user`, set null: the login that did it; null when a customer or the system did), `occurredAt`, `dueAt`, `doneAt` (all `timestampz`, nullable).
- A check: exactly one of `occurredAt` and `dueAt` is set; `doneAt` only when `dueAt` is. A row with `occurredAt` happened; a row with `dueAt` is a next step, done once `doneAt` is set.
- Indexes: `(organizationId, contactId, occurredAt)` for a timeline, and `(organizationId, dueAt)` where `doneAt` is null for the next-step queue.
- `recordActivity(organizationId, { contactId, type, payload?, actorUserId?, occurredAt? })` writes a happened row, `occurredAt` defaulting to now.
- The payload's shape belongs to the feature that writes each type (feature 5 defines `booking_created` 's). Anything from a customer in a payload is rendered as text, never HTML, when feature 11 shows it.

The types are text, never a Postgres enum (project plan: stages, resources and the timeline are tables, never enums). The list lives once in `packages/shared/crm/activity-types.ts`; the database check repeats it, and a new type is one migration and one line.

Testing

- Backend: one test file per function in `backend/lib/crm/`, against the local `scheduleads_dev` with the existing local-dev-database guard, each making and removing its own throwaway businesses.

Database rules (the composite foreign key, the checks, the unique indexes) are proved by inserting the bad row and expecting Postgres to refuse it.

- Shared: the two lists and the contact schema.
- Each new test shown able to fail once, as in earlier features.
- No frontend change, so no browser check.

Notes for the AI

- "The API routes the loop writes to" (build plan) is read as backend functions, not HTTP routes: every write in features 5 to 8 happens inside the API (the public booking route, the job runner), and no screen calls these until feature 11. An HTTP route with no caller is a route to secure and test for nothing. Frank confirms this reading in the whole-feature pass.
- Migrations: generate from `packages/shared` only, read the SQL before committing (the standards), and the 0006 backfill is hand-written at the end of the generated file, as 0004's was. Apply to the local database only; Railway gets them at deploy.
- The hook runs outside Better Auth's writes; `provision-client.ts` already removes a business when anything after its save fails, the stages included.
- The overview's data model passage lists `contact.email` without saying whether it is required; this spec makes it optional on purpose (owner-made phone bookings in feature 5).

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `crm/` is a new code folder with its own import path (`@scheduleads-app/shared/crm` , through `crm/index.ts`), listed in both TypeScript setups as the standards require. It holds the two fixed lists: `default-pipeline-stages.ts` (New, Contacted, Booked, Done) and `activity-types.ts` (the eight timeline types).
- `db/crm-tables/pipeline-stage-table.ts` : name and position per business. Names are unique ignoring capitals (an index on `lower(name)` , Frank's tightening), positions are not unique so later reordering needs no shuffling, and a unique `(organizationId, id)` lets feature 5's lead point only at a stage of its own business. Migration 0006 creates it and, by a hand-written last statement, gives every existing business the four.
- `db/crm-tables/contact-table.ts` : name, email, phone. A partial unique index on `(organizationId, email)` is what makes "the same email in the same business is the same contact" hold even for two bookings at the same instant; a check refuses an email not stored

lowercase and trimmed. Email and phone are both optional (a phone booking in feature 5 may have no email). Migration 0007.

- `db/crm-tables/activity-table.ts` : the timeline and the next-step queue in one table. A composite foreign key to `contact(organizationId, id)` keeps a row inside its business and deletes it with its contact; checks refuse a type outside the shared list (the check is generated from that list), a row both happened and due or neither, and done without due; the actor's login is set to null when the login is removed, keeping the entry. Two indexes: a contact's timeline, and the open next steps. Migration 0008.
- `zod-validation/crm-validation-schemas/contact-validation-schema.ts` : name 1 to 120, the shared email rule, phone up to 40.
- `scripts/seed-dev.ts` gives each dev business the four stages when it has none, as the create hook does.

backend

- `lib/crm/seed-pipeline-stages.ts` : the four stages for a business with none; one with any keeps its own; two calls at once still give one of each (the name index and `on conflict do nothing`). Called by the `afterCreateOrganization` hook right after the first person, so a client set up through `POST /admin/clients` gets them; if it fails, 3b's clean-up removes the half-made business.
- `lib/crm/find-first-pipeline-stage.ts` : the lowest position, ties by `createdAt` then `id`, inside the business. Where feature 5 puts a new lead.
- `lib/crm/find-or-create-contact.ts` : tries to save first and reads only when that changed nothing, so the database, not timing, decides; the lookup is limited to the business; the first name and phone given are kept. Any database error leaves as "Saving a contact failed: database error" and its code, never the query, because the query carries the customer's email (Frank's tightening, the lesson of 3b's F-41).
- `lib/crm/record-activity.ts` : records something that happened (now unless given, by no one unless an actor is named), and lets no payload out in an error, since a payload can hold a customer's words.
- Tests sit beside each function and run against the local database. Every test makes its own businesses and contacts (F-53, F-54), so each passes on its own. The database rules are proved by expecting Postgres to refuse the bad row with the named constraint.

Carried on the branch

- Build-plan item 12c (backups, restore and retention, before Primo) and the matching project-plan bullet, added on Frank's call while planning this feature.
- The pending findings cleared before step 4.2: three out-of-date comments and the coding standards' platform-admin pointer (F-52), history removed from three config comments (F-34), and the sign-in form moved onto the Forms standard: `auth-card.tsx` 's `Field` is now shadow `Input` and `Label`,

both sign-in steps are react-hook-form forms through `Controller` , and the Set up a client form shares the same `Field` (F-14).

Not done here, on purpose

- No web routes: the build plan's "API routes the loop writes to" was read, with Frank, as backend functions; routes come with their screens (features 11 and 12).
- Notes for feature 5: `activity` will likely need a lead link (a contact with two jobs); a form sends no email rather than an empty one; `recordActivity` does not check that an actor belongs to the business; a public route must not send a found contact's stored name or phone back.
- Migrations 0005 to 0008 are applied to the local database only; Railway gets them at deploy, with the F-47 check for old invitations.
- The overview's plan fingerprint is left as it was: it differs from the plans because of plan edits the overview has not seen (12c among them), and rewriting it would hide that drift. `/overview` refreshes it.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-14 was found in feature 1 and F-34 in feature 2, both resolved here; F-32 and F-47 stay in the live ledger.

4/F-14 [P3] closed - The sign-in and create-business forms bypass the project's form standard without saying so

File: frontend/components/auth-card.tsx:240 **Found:** 2026-09-23 by /audit (scope: current; lens: quality)

Why it matters: `coding-standards.md` (Forms) says to use shadcn `Input` and `Label` and to wire forms with react-hook-form. Both forms in this feature use `useState` and a hand-written `Field` , and `frontend/components/ui/` holds only `button.tsx` . That may well be the right call for two one-field forms, but the workspace rule "Do it the right way, out loud" requires the deviation to be named when it is taken, and neither the spec preamble nor the build log records it. Item 2 onward builds real forms and will copy whichever pattern it finds.

Suggested fix: Either move both forms onto shadcn `Input` / `Label` with react-hook-form (both already dependencies), or record the choice and its reason in the spec preamble and build log so the next form knows which pattern is the standard. **Resolution:** Step 3b.3, 2026-09-30: the create form is deleted, and the new Set up a client form follows the Forms standard (shadcn `Input` and `Label` , react-hook-form's `Controller` , the shared schema through `zodResolver`). Only the sign-in form still uses the hand-written `Field` ; out of feature 3b's scope. Fixed 2026-10-01 on Frank's call (the pending tasks before step 4.2), in a chore commit on `feature/04-crm-spine` ; waits for step 4.2's review to close. The sign-in form now follows the Forms standard: `auth-card.tsx` 's `Field` is built on shadcn `Input` and `Label` , both sign-in steps are react-hook-form forms through `Controller` with the shared schemas via

`zodResolver`, and the Set up a client form uses the same `Field`. Checked in the browser against the local API: a bad email and a short code each showed their message under the field with focus there; "Use a different email" kept the address; a typed "Admin@Example.com" was sent trimmed and lowercased; the dev admin signed in to Summit Painting; the Set up a client form still showed all three messages. Closed 2026-10-01 by /audit independent (scope: step 4.2, re-examining 8fa63a3): `auth-card.tsx`'s `Field` now renders `shadcn Label` and `Input` (Base UI); both sign-in steps and the Set up a client form wire it through `react-hook-form Controller` with `zodResolver` and `noValidate`, and the submit handlers receive the schema's output (the email trimmed and lowercased). No form keeps a hand-written input. `npm run build --workspace=frontend` and `npm run lint --workspace=frontend` pass. No new defect found in the repair.

4/F-34 [P3] closed - Three config comments carry history the comment standard keeps out of code

File: `backend/tsconfig.types.json:2` **Found:** 2026-09-28 by /audit independent (scope: current; lens: quality) **Why it matters:** `coding-standards.md` (Comments, "The balance") says no history in code comments, naming step numbers as the first example: that lives in the build log. This feature added `backend/tsconfig.types.json:2` ("AppType, step 2.5"), `backend/tsconfig.json:14` and `packages/shared/tsconfig.build.json:3` ("No src/ folder (Frank, 2026-09-26)"). The Sep 27 comment sweep covered the code files, not the tsconfigs. Harmless at runtime; it is the pattern the next config file copies. **Suggested fix:** Drop "step 2.5" and the two "(Frank, 2026-09-26)" asides and keep the why ("declarations only, because Vercel builds only the frontend"; "no src/ folder, so the code sits beside the build output"). **Resolution:** Fixed 2026-10-01 on Frank's call (the pending tasks before step 4.2), in a chore commit on `feature/04-crm-spine`; waits for step 4.2's review to close. `backend/tsconfig.types.json` no longer says "step 2.5", and `backend/tsconfig.json` and `packages/shared/tsconfig.build.json` no longer carry "(Frank, 2026-09-26)"; each keeps its why. Closed 2026-10-01 by /audit independent (scope: step 4.2, re-examining 8fa63a3): the three tsconfig comments carry no step number or dated aside and each keeps its reason; both builds pass. The same dated-aside pattern in two new step 4.2 files is a separate finding, F-55.

4/F-52 [P3] closed - Three comments still point at the advisory lock and at a role check that has moved

File: `backend/lib/admin/provision-client.ts:171` **Found:** 2026-09-30 by /audit independent (scope: current; lens: quality) **Why it matters:** This project treats its comments and standards as the record of why, and each of these now sends the reader to something that is not there:

- `provision-client.ts:171` says the address "was free when this setup checked, under its lock". There is no lock any more; the address is held by a `client_setup_claim` row, and that is exactly the reason the business removed in the clean-up can only be this setup's.
- `require-platform-admin-middleware.ts:15` calls itself "The one place a role name is compared", but `provision-client.ts:143` also compares `login.role === "admin"` (to refuse the platform

admin's email), and `frontend/lib/is-platform-admin.ts:5` does the same for the screen.

- `coding-standards.md:402-403` names the platform-admin exception "(see `allowUserToCreateOrganization`)". Since this feature that option is the literal `false` and compares no role; the check lives in `requirePlatformAdminMiddleware`. Harmless at runtime. It is the kind of drift F-12 recorded, and the next item that touches the setup path or the admin role reads these lines first. **Suggested fix:** Say "under its claim" at `:171`; reword the middleware line to "the platform admin is the one role compared by name (coding standards, Backend)"; point the standard at `requirePlatformAdminMiddleware` instead of `allowUserToCreateOrganization`. **Resolution:** Fixed 2026-10-01 on Frank's call (the pending tasks before step 4.2), in a chore commit on `feature/04-crm-spine`; waits for step 4.2's review to close. `provision-client.ts` says "under its claim"; the platform-admin middleware says the platform admin is the one role compared by name, not that it is the one place; `coding-standards.md` points the exception at `requirePlatformAdminMiddleware`. Closed 2026-10-01 by /audit independent (scope: step 4.2, re-examining 8fa63a3): `provision-client.ts:171` says "under its claim", matching the `client_setup_claim` design; `require-platform-admin-middleware.ts:15` says the platform admin is the one role compared by name, which stays true with `provision-client.ts:143` and `frontend/lib/is-platform-admin.ts` comparing it too; `coding-standards.md:402` points at `requirePlatformAdminMiddleware`, where the check lives. No new defect.

4/F-53 [P3] closed - The same-name test passes on any refusal and only after an earlier test has run

File: `backend/lib/crm/seed-pipeline-stages.test.ts:92` **Found:** 2026-09-30 by /audit independent (scope: step 4.1; lens: tests) **Why it matters:** The test that proves one business cannot have "New", "new" or "NEW" twice asserts a bare `rejects.toThrow()`, so any failed insert counts as proof of the unique index (a broken foreign key or a renamed column would pass it too). Every other rejection in the backend tests names the error it expects. It also relies on `first` having been seeded by a test in the other `describe` block: run on its own (`vitest -t "capitals ignored"`), `first` has no stages, the `add("New")` insert succeeds and the test fails for a reason unrelated to the rule. The rule itself is correct today (the index in migration 0006 is `("organizationId", lower("name"))`, and the suite passes). **Suggested fix:** Seed its own throwaway business inside the test, and assert the refusal is the name index, for example `rejects.toMatchObject({ cause: { constraint_name: "pipeline_stage_organization_name_unique" } })` (or the code `23505`), for all three inserts.

Resolution: Carried to step 4.2 on Frank's call, 2026-10-01. Written into that step's plan in the spec. Fixed 2026-10-01 in step 4.2's commit; waits for step 4.2's review to close. The capitals test makes its own business, adds New, then expects each of New, new and NEW to be refused with Postgres code `23505` on `pipeline_stage_organization_name_unique`, not any failure. Closed 2026-10-01 by /audit independent (scope: step 4.2, re-examining 1851c20): the test makes its own `capitals` business (removed by the file's `afterAll` through the run tag), inserts New, and expects New, new and NEW each refused with `cause` code `23505` on `pipeline_stage_organization_name_unique`. Run alone (`npm`

`run test --workspace=backend -- lib/crm/seed-pipeline-stages.test.ts -t "capitals ignored" )` it passes, and the full backend suite passes (141 tests); no test business was left behind.

4/F-54 [P3] closed - The contact tests lean on the first test's row, the order dependence F-53 just removed

File: backend/lib/crm/find-or-create-contact.test.ts:138 **Found:** 2026-10-01 by /audit independent (scope: step 4.2; lens: tests) **Why it matters:** Four tests (:61 , :72 , :82 , and the database rule at :138 , which sits in a different `describe` block) work only because the first test at :47 already made Maria in `primo` . Run on its own, the rule test fails for a reason unrelated to the rule: `npm run test -- workspace=backend -- lib/crm/find-or-create-contact.test.ts -t "cannot hold the same email twice"` inserts the row successfully and fails its `rejects` assertion (reproduced in this review). This is the second half of F-53, fixed in the stage tests in the same commit and reintroduced here, and step 4.3's activity tests will copy whichever pattern they find. The full suite passes today only because Vitest runs a file's tests in order. **Suggested fix:** Let no test depend on another having run: the database rule test makes its own throwaway business and inserts the first `maria@...` row itself before expecting the 23505; the matching tests either do the same or share one `beforeAll` that creates Maria in `primo` . **Resolution:** Fixed 2026-10-01 on Frank's yes, in the step 4.2 review-fix commit; waits for step 4.3's review to close. Every contact test makes its own businesses, and its own Maria through a `businessWithMaria` helper; each of the nine passes run on its own (`-t` one at a time). The three faults planted before (raw error out, lookup ignoring the business, no conflict handling) are each still caught. Closed 2026-10-01 by /audit independent (scope: step 4.3, re-examining fa4c6b3): the shared `primo / clinic` and `beforeAll` are gone; each of the nine tests makes its own throwaway businesses (slugs `new` , `again` , `mine` , `clinic` , `both-primo` , `both-clinic` , `race` , `phone` , `lowercase` , `twice` , all distinct, removed by the tag `afterAll`), and the rule test at :151 makes its own Maria through `businessWithMaria` before expecting 23505 on `contact_organization_email_unique` . Each of the nine run alone with `npm run test --workspace=backend -- lib/crm/find-or-create-contact.test.ts -t "<name>"` passes (1 passed, 8 skipped), and the full backend suite passes (151 tests). No new defect in the repair.

4/F-55 [P3] closed - Two new files put a dated "(Frank, 2026-10-01)" aside in code comments, the history F-34 just removed

File: backend/lib/crm/find-or-create-contact.ts:2 **Found:** 2026-10-01 by /audit independent (scope: step 4.2; lens: quality) **Why it matters:** `coding-standards.md` (Comments, "The balance") keeps history out of code comments, and F-34, closed in this same review, removed exactly this shape ("(Frank, 2026-09-26)") from three tsconfigs. Step 4.2 adds it again at `backend/lib/crm/find-or-create-contact.ts:2` and `packages/shared/db/crm-tables/contact-table.ts:2` . Harmless at runtime; the decision and its date already live in the spec and the build log, and the next table file copies whichever header it finds. **Suggested fix:** Keep the rule and drop the aside, for example "The same email in the same business is the same contact; no email is always new." **Resolution:** Fixed 2026-10-01 on Frank's yes, in the step 4.2 review-fix commit; waits for step 4.3's review to close. The two comments say "the same email in the

same business is the same contact" with no dated aside. Closed 2026-10-01 by /audit independent (scope: step 4.3, re-examining fa4c6b3): `find-or-create-contact.ts:2` and `contact-table.ts:2` keep the rule with no dated aside, and a search of `backend/lib/crm` , `packages/shared/crm` , `packages/shared/db/crm-tables` and `0008_activity.sql` finds no "(Frank, ...)", step or finding number, nor any em dash, in step 4.3's new files either. Both builds pass. No new defect.

4/F-56 [P3] closed - The actor's "set null" rule is the one activity database rule no test proves

File: `packages/shared/db/crm-tables/activity-table.ts:24` **Found:** 2026-10-01 by /audit independent (scope: step 4.3; lens: tests) **Why it matters:** The spec's Testing section asks for a test of every database rule, and step 4.3 proves all of them except `activity_actorUserId_user_id_fk ... ON DELETE set null` . That rule is what keeps a customer's timeline when the login that wrote an entry is removed (a staff member leaving): if it were ever generated as `cascade` , removing a login would silently delete timeline rows, and with Postgres's default `no action` the login could not be removed at all. `record-activity.test.ts:49` deletes its test user only after the businesses (and their timelines) are already gone, so the rule is never exercised. **Suggested fix:** One test in `record-activity.test.ts` : record an entry with a throwaway actor, delete that user, and expect the entry still on the timeline with `actorUserId: null` . **Resolution:** Fixed 2026-10-01 on Frank's yes, in the step 4.3 review-fix commit; waits for the final review at /complete to close. `record-activity.test.ts` records a call by a login, deletes that login, and expects the entry kept with `actorUserId: null` ; it passes run on its own, and the live constraint reads ON DELETE SET NULL. Not proved by breaking it: that would mean altering the local database's foreign key; with cascade the entry would vanish and with no action the delete would be refused, both of which the test catches. The review's note on the drift test is answered too: `activity-types.ts` now says only what the test proves. Closed 2026-10-01 by /audit independent (scope: current, final review of feature 4): `record-activity.test.ts:176` ("removing a login keeps the entries it made, with no one named") records a call by a throwaway login, deletes that login, and expects the one entry kept with `actorUserId: null` ; run alone (`npm run test --workspace=backend -- lib/crm/record-activity.test.ts -t "removing a login"`) it passes (1 passed, 10 skipped), and the full backend suite passes (152 tests). Migration 0008 declares `ON DELETE set null` , and a read-only query of the local database's `pg_constraint` shows `activity_actorUserId_user_id_fk` with `confdeltype n` (SET NULL). A `cascade` rule would leave the timeline empty and a `no action` rule would refuse the delete, so the test fails either way. The `activity-types.ts` header claims only what the drift test proves. No new defect in the repair.

Independent review

Status: passed **Target commit:** `cbdfb55bfef97aba10a1953086c47c0cd5066c73` **Base commit:** `4b117eb438af4d1c590aaa3e28be06dfba4f8451` **Base ref:** main **Spec hash:** `fb502442b77172f4573a733740ec1e1f9ce7a606bd7be46492332238d2bc332c` **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5

Requested execution: automatic **Requested at:** 2026-10-01T16:22:05Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-01T16:25:15Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`4b117eb438af4d1c590aaa3e28be06dfba4f8451..cbdfb55bfef97aba10a1953086c47c0cd5066c73` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `git rev-parse HEAD`, `git merge-base main HEAD`, `sha256sum blueprint/context/current-feature.md`, `git status --porcelain` : pass (HEAD, merge base and spec hash match the request; only review.md differed)
- `npm run test --workspace=@scheduleads-app/shared` : pass (9 files, 66 tests)
- `npm run test --workspace=backend` : pass (11 files, 152 tests, against the local scheduleads_dev)
- `npm run test --workspace=backend -- lib/crm/record-activity.test.ts -t "removing a login"` : pass (1 passed, 10 skipped)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass
- `npm run lint --workspace=frontend` : pass
- Read-only probes: the local database's `pg_constraint` for `activity`, leftover `test-%` businesses, stages per business; the built contact schema's error messages and a non-ASCII email: pass, nothing written

Evidence

- All eight commits on the branch reviewed (`c47954c` plan item 12c, `cfff265` 4.1, `8fa63a3` pending-findings chore, `1851c20` 4.2, `fa4c6b3` and `749e9fa` review fixes, `31752e9` 4.3, `cbdfb55` spec verified).
- Tenancy: every new table carries `organizationId` (FK, cascade); `findOrCreateContact` reads by `(organizationId, email)` only; `activity_contact_fk` is the composite `(organizationId, contactId)` to `contact(organizationId, id)`, and the cross-business test is refused with 23503 by the database. No HTTP route added, so no new trust boundary.

- Migrations 0006 to 0008 match the schema files and the 0008 snapshot (indexes, unique constraints, three activity checks, three foreign keys). The 0006 backfill is an insert-only `CROSS JOIN` of the four defaults over existing businesses into a table created in the same file; locally both dev businesses hold four stages.
- Concurrency: `findOrCreateContact` inserts `on conflict do nothing` then reads, and `seedPipelineStages` relies on the case-insensitive name index; both race tests run on a 10-connection pool, so they really overlap.
- Personal data in errors: both CRM functions rethrow only `safeErrorReason` (Postgres code); the contact schema's `ZodError` carries messages, not input (probed with an invalid email); `z.email` refuses non-ASCII, so JS lowercasing cannot disagree with `contact_email_normalized_check`.
- The create hook seeds stages after the first person; a failure falls into `provision-client.ts`'s existing clean-up, and `admin-routes.test.ts` proves a set-up client has New, Contacted, Booked, Done.
- Sign-in chore: both steps use react-hook-form `Controller` with `zodResolver` and `noValidate`; the success path (`router.push("/")` , `router.refresh()`) and the enumeration-safe messages are unchanged; `Field` now spreads props before `id` and the aria attributes, so a caller cannot override them.
- No focused, skipped or placeholder tests in the new test files; no em dash, dated aside or step number in new code comments; no AI attribution in the commit messages; no test business left in the local database.

Findings

- F-56 [P3] closed by this review (re-examined, defect gone, no new defect).
- No new findings. F-32 and F-47 left as they were (carried P3, outside this delta's code).

Remaining risk

- No `Verify` command and no GitHub check are declared, so there is no single umbrella gate; the five commands above were run individually.
- The frontend has no test runner and no browser tests exist; the sign-in form rework is proven by build, lint and the builder's recorded manual browser check, not by a test this review could rerun. Check was not required and was not run.
- Migration snapshots were checked by reading, not by `db:generate` (not run, to keep the working tree clean), so a schema-to-snapshot drift outside the three new tables would not have been seen.
- For feature 5, not defects today: `recordActivity` does not check that `actorUserId` is a member of the business (the caller must pass the session's own login); a public booking route must not echo a found contact's stored name or phone back to the caller, because `findOrCreateContact` returns the

existing contact for any matching email; and a form that sends an empty email string gets a validation error rather than "no email".